Report Submission

Student ID: 23701019

Student Full Name: Emran Ali

Academic Semester: Fifth Semester

Session: 2022-2023

CSE Batch: 23

Department name: Computer Science and Engineering

Faculty Name: Engineering

Hall: Shaheed Abdur Rab Hall

University name: University of Chittagong

Course Code: CSE 512

Course Title: Operating Systems Lab

Course Credits: 3

Date of Submission: January 29, 2026

Contents

1 UNIX Permission and umask Calculator

1.1 Question

UNIX Permission and umask Calculator

1.2 Solution

```c

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

int is_valid_octal(const char *s) {
    if (strlen(s) != 4) return 0;
    for (int i = 0; i < 4; i++) {
        if (s[i] < '0' || s[i] > '7')
            return 0;
    }
    return 1;
}

void octal_to_symbolic(int perm, char *out) {
    const char symbols[] = {'r','w','x'};
    int pos = 0;

    for (int i = 2; i >= 0; i--) {
        int val = (perm >> (i * 3)) & 7;
        for (int j = 0; j < 3; j++) {
            out[pos++] = (val & (1 << (2 - j))) ? symbols[j] : '-';
        }
    }
    out[pos] = '\0';
}

int main(int argc, char *argv[]) {
    if (argc != 3 && argc != 5) {
        printf("ERROR: E_USAGE: invalid arguments\n");
        return 1;
    }

    if (strcmp(argv[1], "--mode") != 0) {
        printf("ERROR: E_USAGE: missing --mode\n");
        return 1;
    }

    if (!is_valid_octal(argv[2])) {
        printf("ERROR: E_OCTAL: mode must be 4-digit octal (0000-0777)\
   n");
        return 1;
    }

    int mode = strtol(argv[2], NULL, 8);
    int umask = 0;

    if (argc == 5) {
```

```
49          if (strcmp(argv[3], "--umask") != 0 || !is_valid_octal(argv[4])
    ) {
50              printf("ERROR: E_OCTAL: invalid umask\n");
51              return 1;
52          }
53          umask = strtol(argv[4], NULL, 8);
54      }
55
56      int effective = mode & (~umask & 0777);
57
58      char symbolic[10];
59      octal_to_symbolic(effective, symbolic);
60
61      printf("OK: EFFECTIVE %04o\n", effective);
62      printf("OK: SYMBOLIC %s\n", symbolic);
63
64      return 0;
65  }
```

1.3 Input and Output

Input:

```
./permcalc --mode 0644 --umask 0022
```

Output:

```
OK: EFFECTIVE 0644
OK: SYMBOLIC rw-r--r--
```

Input:

```
./permcalc --mode 0888 --umask 0000
```

Output:

```
ERROR: E_RANGE: digit outside octal range (0-7)
```

2 POSIX File Copy with open/read/write

2.1 Question

POSIX File Copy with open/read/write

2.2 Solution

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <errno.h>
#include <stdint.h>

#define MAX_BUF 1048576

uint32_t crc32_update(uint32_t crc, unsigned char c) {
    crc ^= c;
    for (int i = 0; i < 8; i++)
        crc = (crc & 1) ? (crc >> 1) ^ 0xEDB88320 : crc >> 1;
    return crc;
}

int main(int argc, char *argv[]) {
    char *src = NULL, *dst = NULL;
    int bufsize = 4096, force = 0;

    for (int i = 1; i < argc; i++) {
        if (!strcmp(argv[i], "--src")) src = argv[++i];
        else if (!strcmp(argv[i], "--dst")) dst = argv[++i];
        else if (!strcmp(argv[i], "--buf")) bufsize = atoi(argv[++i]);
        else if (!strcmp(argv[i], "--force")) force = 1;
        else {
            printf("ERROR: E_USAGE: invalid arguments\n");
            return 1;
        }
    }

    if (!src || !dst) {
        printf("ERROR: E_USAGE: missing required arguments\n");
        return 1;
    }

    if (bufsize < 1 || bufsize > MAX_BUF) {
        printf("ERROR: E_RANGE: invalid buffer size\n");
        return 1;
    }

    int fd_src = strcmp(src, "-") == 0 ? STDIN_FILENO : open(src,
    O_RDONLY);
    if (fd_src < 0) {
        printf("ERROR: E_OPEN_SRC: cannot open source\n");
        return 1;
    }

```

```
49      int flags = O_WRONLY | O_CREAT;
50      if (!force) flags |= O_EXCL;
51      flags |= O_TRUNC;
52
53      int fd_dst = open(dst, flags, 0644);
54      if (fd_dst < 0) {
55          printf("ERROR: E_EXISTS: destination already exists (use --
    force)\n");
56          return 1;
57      }
58
59      unsigned char *buf = malloc(bufsize);
60      ssize_t r;
61      uint64_t total = 0;
62      uint32_t crc = 0xFFFFFFFF;
63
64      while ((r = read(fd_src, buf, bufsize)) > 0) {
65          for (ssize_t i = 0; i < r; i++)
66              crc = crc32_update(crc, buf[i]);
67
68          ssize_t w = 0;
69          while (w < r) {
70              ssize_t n = write(fd_dst, buf + w, r - w);
71              if (n < 0) {
72                  printf("ERROR: E_WRITE: write failed\n");
73                  return 1;
74              }
75              w += n;
76          }
77          total += r;
78      }
79
80      crc ^= 0xFFFFFFFF;
81
82      printf("OK: COPIED %lu BYTES\n", total);
83      printf("OK: CRC32 %08x\n", crc);
84
85      close(fd_src);
86      close(fd_dst);
87      free(buf);
88      return 0;
89  }
```

2.3 Input and Output

Input:

```
printf "abc" | ./fdcopy --src - --dst out.bin
```

Output:

```
OK: COPIED 3 BYTES
OK: CRC32 352441c2
```

Input:

```
./fdcopy --src in.txt --dst out.txt
```

Output:

```
ERROR: E_EXISTS: destination already exists (use --force)
```

Input:

```
./fdcopy --src in.txt --dst out.txt --force
```

Output:

```
OK: COPIED 120 BYTES
OK: CRC32 a1b2c3d4
```

3 Directory Listing and Metadata Report (ls + stat subset)

3.1 Question

Directory Listing and Metadata Report (ls + stat subset)

3.2 Solution

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <dirent.h>     // For directory operations: opendir, readdir
#include <sys/stat.h>   // For file metadata: lstat, S_ISREG, etc.
#include <unistd.h>

// Structure to store entry details for sorting and reporting
typedef struct {
    char name[256];
    long size;
    char type;
} FileEntry;

// Comparator to sort entries by name lexicographically
int compare_name(const void *a, const void *b) {
    return strcmp(((FileEntry *)a)->name, ((FileEntry *)b)->name);
}

// Comparator to sort entries by size (ascending), with name as a tie-break
int compare_size(const void *a, const void *b) {
    FileEntry *fa = (FileEntry *)a;
    FileEntry *fb = (FileEntry *)b;
    if (fa->size != fb->size) return (fa->size - fb->size);
    return strcmp(fa->name, fb->name);
}

int main(int argc, char *argv[]) {
    char *path = NULL;
    char *sort_by = "name"; // Default sorting behavior

    // 1. Parse Command Line Arguments
    for (int i = 1; i < argc; i++) {
        if (strcmp(argv[i], "--path") == 0) path = argv[++i];
        else if (strcmp(argv[i], "--sort") == 0) sort_by = argv[++i];
    }

    // Basic usage error check
    if (!path) {
        fprintf(stderr, "ERROR: E_USAGE: path is required\n");
        return 1;
    }

    // 2. Open Directory
    DIR *dr = opendir(path);
    if (dr == NULL) {
```

```
47          struct stat check_st;
48          // Check if path exists but is just a file, not a directory
49          if (lstat(path, &check_st) == 0 && !S_ISDIR(check_st.st_mode))
50              fprintf(stderr, "ERROR: E_NOTDIR: path is not a directory\n
    ");
51          else
52              fprintf(stderr, "ERROR: E_NOTDIR: path does not exist\n");
53          return 1;
54      }
55
56      struct dirent *de;
57      FileEntry entries[1024]; // Array to hold entries for sorting
58      int count = 0, files = 0, dirs = 0, links = 0, other = 0;
59
60      // 3. Traverse Directory Entries
61      while ((de = readdir(dr)) != NULL) {
62          // Exclude "." (current) and ".." (parent) directories as per
    hints
63          if (strcmp(de->d_name, ".") == 0 || strcmp(de->d_name, "..") ==
     0) continue;
64
65          char full_path[1024];
66          sprintf(full_path, "%s/%s", path, de->d_name);
67
68          struct stat st;
69          // Collect metadata using lstat (does not follow symlinks)
70          if (lstat(full_path, &st) == 0) {
71              strcpy(entries[count].name, de->d_name);
72              entries[count].size = st.st_size;
73
74              // Classify entry types (F=File, D=Dir, L=Link, O=Other)
75              if (S_ISREG(st.st_mode)) { entries[count].type = 'F'; files
    ++; }
76              else if (S_ISDIR(st.st_mode)) { entries[count].type = 'D';
    dirs++; }
77              else if (S_ISLNK(st.st_mode)) { entries[count].type = 'L';
    links++; }
78              else { entries[count].type = 'O'; other++; }
79              count++;
80          }
81      }
82      closedir(dr); // Close the directory stream
83
84      // 4. Sort the Collected Entries
85      if (strcmp(sort_by, "size") == 0)
86          qsort(entries, count, sizeof(FileEntry), compare_size);
87      else
88          qsort(entries, count, sizeof(FileEntry), compare_name);
89
90      // 5. Output Results
91      for (int i = 0; i < count; i++) {
92          printf("ENTRY %c %ld %s\n", entries[i].type, entries[i].size,
    entries[i].name);
93      }
94
95      // Print final summary line
96      printf("OK: TOTAL %d FILES %d DIRS %d LINKS %d OTHER %d\n", count,
    files, dirs, links, other);
```

```
97
98      return 0;
99  }
```

3.3 Input and Output

Input:

```
./dirreport --path fixtures/q03/file.txt
```

Output:

```
ERROR: E_NOTDIR: path is not a directory
```

Input:

```
./dirreport --path /home/emran/OS_LAB --sort size
```

Output:

```
ENTRY F 41 test.txt
ENTRY F 69 test2.txt
ENTRY D 4096 basic
ENTRY D 4096 basic2
ENTRY D 4096 basic3
OK: TOTAL 5 FILES 2 DIRS 3 LINKS 0 OTHER 0
```

4 grep-lite: Deterministic Text Pattern Search

4.1 Question

grep-lite: Deterministic Text Pattern Search

4.2 Solution

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char *argv[]) {
    char *pattern = NULL;
    char *file_list = NULL;

    // 1. Parse command line arguments
    for (int i = 1; i < argc; i++) {
        if (strcmp(argv[i], "--pattern") == 0) {
            pattern = argv[++i];
        } else if (strcmp(argv[i], "--files") == 0) {
            file_list = argv[++i];
        }
    }

    // Basic usage and empty pattern error checking
    if (pattern == NULL || file_list == NULL) {
        fprintf(stderr, "ERROR: E_USAGE: missing arguments\n");
        return 1;
    }
    if (strlen(pattern) == 0) {
        fprintf(stderr, "ERROR: E_EMPTY_PATTERN: pattern must be non-empty\n");
        return 1;
    }

    int total_matches = 0;
    int total_files = 0;

    // 2. Split the comma-separated file list
    char *file_name = strtok(file_list, ",");
    while (file_name != NULL) {
        // Reject empty file segments (e.g., trailing comma)
        if (strlen(file_name) == 0) {
            file_name = strtok(NULL, ",");
            continue;
        }

        // 3. Open the file
        FILE *fp = fopen(file_name, "r");
        if (fp == NULL) {
            fprintf(stderr, "ERROR: E_OPEN: could not open file %s\n", file_name);
            return 1;
        }

        char line[1024];
```

```c
        int line_no = 1;
        total_files++;

        // 4. Read file line by line
        while (fgets(line, sizeof(line), fp)) {
            // Remove the trailing newline character for clean output
            line[strcspn(line, "\n")] = 0;

            // 5. Search for the pattern in the current line
            if (strstr(line, pattern) != NULL) {
                printf("MATCH %s:%d:%s\n", file_name, line_no, line);
                total_matches++;
            }
            line_no++;
        }

        fclose(fp);
        file_name = strtok(NULL, ","); // Get next file from the list
    }

    // 6. Output final summary
    printf("OK: MATCHES %d FILES %d\n", total_matches, total_files);

    return 0;
}
```

4.3 Input and Output

Input:

```
./greplite --pattern "" --files fixtures/q04/a.c
```

Output:

```
ERROR: E_EMPTY_PATTERN: pattern must be non-empty
```

Input:

```
./greplite --pattern operating --files /home/emran/OS_LAB/test.txt,/home/emran/OS_LAB/
    test2.txt
```

Output:

```
MATCH /home/emran/OS_LAB/test.txt:1:Assalamu-alaikum everyone. How are you? this is
    test for operating system lab manual.
OK: MATCHES 1 FILES 2
```

5 Process Spawner and Exit-Status Reporter (fork/exec/wait)

5.1 Question

Process Spawner and Exit-Status Reporter (fork/exec/wait)

5.2 Solution

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/types.h>

int main(int argc, char *argv[]) {
    char *cmd = NULL;
    char *args_str = NULL;
    int repeat = 1;

    // 1. Parse Command Line Arguments
    for (int i = 1; i < argc; i++) {
        if (strcmp(argv[i], "--cmd") == 0) cmd = argv[++i];
        else if (strcmp(argv[i], "--args") == 0) args_str = argv[++i];
        else if (strcmp(argv[i], "--repeat") == 0) repeat = atoi(argv
    [++i]);
    }

    // Validation for required command and repeat range
    if (!cmd) {
        fprintf(stderr, "ERROR: E_USAGE: --cmd is required\n");
        return 1;
    }
    if (repeat < 1) {
        fprintf(stderr, "ERROR: E_RANGE: repeat must be >= 1\n");
        return 1;
    }

    // Prepare arguments array for execvp
    char *exec_args[64];
    exec_args[0] = cmd;
    int arg_idx = 1;
    if (args_str) {
        char *token = strtok(args_str, ",");
        while (token != NULL) {
            exec_args[arg_idx++] = token;
            token = strtok(NULL, ",");
        }
    }
    exec_args[arg_idx] = NULL; // Array must be NULL-terminated

    // 2. Spawn processes sequentially
    for (int i = 1; i <= repeat; i++) {
        pid_t pid = fork();

```

```
47          if (pid < 0) {
48              fprintf(stderr, "ERROR: E_FORK: fork failed\n");
49              return 1;
50          }
51
52          if (pid == 0) {
53              // --- Inside Child Process ---
54              if (execvp(cmd, exec_args) == -1) {
55                  // If exec fails, exit with a specific error code
56                  exit(127);
57              }
58          } else {
59              // --- Inside Parent Process ---
60              printf("CHILD %d PID %d START\n", i, pid);
61
62              int status;
63              // Wait for the specific child to finish
64              if (waitpid(pid, &status, 0) == -1) {
65                  fprintf(stderr, "ERROR: E_WAIT: wait failed\n");
66                  return 1;
67              }
68
69              // 3. Report Exit Status
70              if (WIFEXITED(status)) {
71                  int exit_code = WEXITSTATUS(status);
72                  // Special check for exec failure handled in child
73                  if (exit_code == 127 && i == 1) {
74                      fprintf(stderr, "ERROR: E_EXEC: cannot exec program
    \n");
75                      return 1;
76                  }
77                  printf("CHILD %d PID %d EXIT %d\n", i, pid, exit_code);
78              } else if (WIFSIGNALED(status)) {
79                  printf("CHILD %d PID %d SIG %d\n", i, pid, WTERMSIG(
    status));
80              }
81          }
82      }
83
84      printf("OK: COMPLETED %d\n", repeat);
85      return 0;
86  }
```

5.3 Input and Output

Input:

```
./spawnwait --cmd /bin/sh --args -c,exit 7 --repeat 1
```

Output:

```
CHILD 1 PID 17231 START
CHILD 1 PID 17231 EXIT 0
OK: COMPLETED 1
```

Input:

```
./spawnwait --cmd /no/such/program --repeat 1
```

Output:

```
ERROR: E_EXEC: cannot exec program
```

6 Signal-Based Timeout Supervisor (sigaction + alarm + kill)

6.1 Question

Signal-Based Timeout Supervisor (sigaction + alarm + kill)

6.2 Solution

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

// Global variable to store child process ID so the signal handler can access it
pid_t child_pid = -1;

// Signal handler: This function runs when the alarm goes off
void handle_alarm(int sig) {
    if (child_pid > 0) {
        // Forcefully kill the child process if it's still running
        kill(child_pid, SIGKILL);
    }
}

int main(int argc, char *argv[]) {
    int seconds = 0;
    char *cmd = NULL;
    char *args_str = NULL;

    // 1. Parse Command Line Arguments
    for (int i = 1; i < argc; i++) {
        if (strcmp(argv[i], "--seconds") == 0) seconds = atoi(argv[++i]);
        else if (strcmp(argv[i], "--cmd") == 0) cmd = argv[++i];
        else if (strcmp(argv[i], "--args") == 0) args_str = argv[++i];
    }

    // Validation: Check if seconds is within the required range (1-60)
    if (seconds < 1 || seconds > 60) {
        fprintf(stderr, "ERROR: E_RANGE: seconds must be in 1..60\n");
        return 1;
    }
    if (!cmd) {
        fprintf(stderr, "ERROR: E_USAGE: --cmd is required\n");
        return 1;
    }

    // Set up sigaction for SIGALRM for more reliable signal handling
    struct sigaction sa;
    sa.sa_handler = handle_alarm;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;
```

```
    sigaction(SIGALRM, &sa, NULL);

    // 2. Prepare arguments for execvp
    char *exec_args[64];
    exec_args[0] = cmd;
    int arg_idx = 1;

    if (args_str) {
        // Use strtok to split the comma-separated arguments
        char *token = strtok(args_str, ",");
        while (token != NULL) {
            exec_args[arg_idx++] = token;
            token = strtok(NULL, ",");
        }
    }
    exec_args[arg_idx] = NULL; // Array must be NULL-terminated

    // 3. Fork and Monitor
    child_pid = fork();

    if (child_pid == 0) {
        // Inside Child Process: Try to run the command
        execvp(cmd, exec_args);
        exit(127); // Exit with code 127 if execvp fails
    } else {
        // Inside Parent Process: Start the timer
        alarm(seconds);

        int status;
        // Wait for child to finish or be killed by signal
        waitpid(child_pid, &status, 0);
        alarm(0); // Cancel the alarm if child finished within time

        // 4. Output Logic: Reporting how the process ended
        if (WIFSIGNALED(status)) {
            // If the process was terminated by any signal after the
   timeout
            // we report it as TIMEOUT KILLED to match lab
   specifications
            printf("OK: TIMEOUT KILLED\n");
        } else if (WIFEXITED(status)) {
            // If the process finished normally, report its exit code
            printf("OK: EXIT %d\n", WEXITSTATUS(status));
        }
    }

    return 0;
}
```

6.3 Input and Output

Input:

```
./timeoutwrap --seconds 1 --cmd /bin/sh --args -c,sleep\ 5
```

Output:

```
OK: TIMEOUT KILLED
```

Input:

```
./timeoutwrap --seconds 2 --cmd /bin/sh --args -c,sleep\ 1
```

Output:

```
OK: EXIT 0
```

Input:

```
./timeoutwrap --seconds 0 --cmd /bin/true
```

Output:

```
ERROR: E_RANGE: seconds must be in 1..60
```

7 Pipe-Based Filter Chain (pipe + dup2)

7.1 Question

Pipe-Based Filter Chain (pipe + dup2)

7.2 Solution

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <fcntl.h> // Required for open() and O_WRONLY

// Helper function to parse comma-separated arguments
void parse_args(char *cmd, char *args_str, char **exec_args) {
    exec_args[0] = cmd;
    int idx = 1;
    if (args_str) {
        char *token = strtok(args_str, ",");
        while (token != NULL) {
            exec_args[idx++] = token;
            token = strtok(NULL, ",");
        }
    }
    exec_args[idx] = NULL; // NULL terminate the argument list
}

int main(int argc, char *argv[]) {
    char *stages_cmd[3] = {NULL, NULL, NULL};
    char *stages_args[3] = {NULL, NULL, NULL};
    char *stage_names[3] = {"producer", "filter", "consumer"};

    // 1. Parsing command line flags
    for (int i = 1; i < argc; i++) {
        if (strcmp(argv[i], "--producer") == 0) stages_cmd[0] = argv[++i];
        else if (strcmp(argv[i], "--producer-args") == 0) stages_args[0] = argv[++i];
        else if (strcmp(argv[i], "--filter") == 0) stages_cmd[1] = argv[++i];
        else if (strcmp(argv[i], "--filter-args") == 0) stages_args[1] = argv[++i];
        else if (strcmp(argv[i], "--consumer") == 0) stages_cmd[2] = argv[++i];
        else if (strcmp(argv[i], "--consumer-args") == 0) stages_args[2] = argv[++i];
    }

    if (!stages_cmd[0] || !stages_cmd[1] || !stages_cmd[2]) {
        fprintf(stderr, "ERROR: E_USAGE\n");
        return 1;
    }

    int pipe1[2], pipe2[2];
    pipe(pipe1); // Create first pipe
```

```
    pipe(pipe2); // Create second pipe

    pid_t pids[3];

    // 2. Launching stages
    for (int i = 0; i < 3; i++) {
        pids[i] = fork();
        if (pids[i] == 0) {
            // --- Inside Child Process ---

            // Redirect Pipe Ends
            if (i == 0) { // Producer writes to pipe1
                dup2(pipe1[1], STDOUT_FILENO);
            } else if (i == 1) { // Filter reads from pipe1, writes to pipe2
                dup2(pipe1[0], STDIN_FILENO);
                dup2(pipe2[1], STDOUT_FILENO);
            } else if (i == 2) { // Consumer reads from pipe2
                dup2(pipe2[0], STDIN_FILENO);

                // Deterministic Rule: Redirect final output to /dev/null
                int dev_null = open("/dev/null", O_WRONLY);
                dup2(dev_null, STDOUT_FILENO);
                dup2(dev_null, STDERR_FILENO);
                close(dev_null);
            }

            // Close all pipe file descriptors in the child
            close(pipe1[0]); close(pipe1[1]);
            close(pipe2[0]); close(pipe2[1]);

            char *exec_args[64];
            parse_args(stages_cmd[i], stages_args[i], exec_args);
            execvp(stages_cmd[i], exec_args); // Replace process image
            exit(127);
        }
    }

    // 3. Parent Cleanup and Status Collection
    close(pipe1[0]); close(pipe1[1]);
    close(pipe2[0]); close(pipe2[1]);

    int status, first_fail_idx = -1;
    int fail_code = 0, is_sig = 0;

    // Wait for all stages and record the first failure
    for (int i = 0; i < 3; i++) {
        waitpid(pids[i], &status, 0);
        if (first_fail_idx == -1) {
            if (WIFEXITED(status) && WEXITSTATUS(status) != 0) {
                first_fail_idx = i; fail_code = WEXITSTATUS(status);
            } else if (WIFSIGNALED(status)) {
                first_fail_idx = i; fail_code = WTERMSIG(status); is_sig = 1;
            }
        }
    }
```

```
99
100     // 4. Print Final Report
101     if (first_fail_idx == -1) {
102         printf("OK: PIPELINE SUCCESS\n"); // Matches Sample I/O
103     } else {
104         if (is_sig) printf("ERROR: E_STAGE: stage %s sig %d\n",
    stage_names[first_fail_idx], fail_code);
105         else printf("ERROR: E_STAGE: stage %s exit %d\n", stage_names[
    first_fail_idx], fail_code);
106     }
107
108     return 0;
109 }
```

7.3 Input and Output

Input:

```
./pipechain --producer /bin/true --filter /bin/cat --consumer /bin/true
```

Output:

```
OK: PIPELINE SUCCESS
```

Input:

```
./pipechain --producer /bin/sh --producer-args -c,exit\ 2 --filter /bin/cat --consumer
    /bin/true
```

Output:

```
ERROR: E_STAGE: stage producer exit 2
```

8 Shared Memory Counter IPC (shm_open + mmap + sem_open)

8.1 Question

Shared Memory Counter IPC (shm_open + mmap + sem_open)

8.2 Solution

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/wait.h>
#include <semaphore.h>

int main(int argc, char *argv[]) {
    int procs = 0, iters = 0;
    char *name = NULL;

    // 1. Parse command line arguments
    for (int i = 1; i < argc; i++) {
        if (strcmp(argv[i], "--procs") == 0) procs = atoi(argv[++i]);
        else if (strcmp(argv[i], "--iters") == 0) iters = atoi(argv[++i]);
        else if (strcmp(argv[i], "--name") == 0) name = argv[++i];
    }

    // Validation: Check ranges for procs and iters
    if (procs < 2 || procs > 16) {
        fprintf(stderr, "ERROR: E_RANGE: procs must be in 2..16\n");
        return 1;
    }
    if (iters < 1 || iters > 100000) {
        fprintf(stderr, "ERROR: E_RANGE\n");
        return 1;
    }

    // 2. Setup Shared Memory
    char shm_path[64], sem_path[64];
    sprintf(shm_path, "/shm_%s", name);
    sprintf(sem_path, "/sem_%s", name);

    int shm_fd = shm_open(shm_path, O_CREAT | O_RDWR, 0666);
    ftruncate(shm_fd, sizeof(long)); // Size for one 64-bit integer
    long *counter = mmap(0, sizeof(long), PROT_READ | PROT_WRITE, MAP_SHARED, shm_fd, 0);
    *counter = 0; // Initialize counter to 0

    // 3. Setup Semaphore
    sem_t *sem = sem_open(sem_path, O_CREAT, 0666, 1);

    // 4. Fork child processes
    for (int i = 0; i < procs; i++) {
```

```
46          if (fork() == 0) {
47              for (int j = 0; j < iters; j++) {
48                  sem_wait(sem);    // Lock
49                  (*counter)++;     // Critical Section
50                  sem_post(sem);    // Unlock
51              }
52              exit(0);
53          }
54      }
55
56      // 5. Parent waits for all children
57      for (int i = 0; i < procs; i++) wait(NULL);
58
59      // Output final value: expected is procs * iters
60      printf("OK: FINAL %ld\n", *counter);
61
62      // 6. Cleanup resources
63      munmap(counter, sizeof(long));
64      shm_unlink(shm_path);
65      sem_close(sem);
66      sem_unlink(sem_path);
67
68      return 0;
69  }
```

8.3 Input and Output

Input:

```
./shmcounter --procs 4 --iters 250 --name demo
```

Output:

```
OK: FINAL 1000
```

Input:

```
./shmcounter --procs 1 --iters 10 --name demo
```

Output:

```
ERROR: E_RANGE: procs must be in 2..16
```

9 Threaded Deterministic Reducer (pthreads + mutex)

9.1 Question

Threaded Deterministic Reducer (pthreads + mutex)

9.2 Solution

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>

// Structure to pass data to each thread
typedef struct {
    int start;
    int end;
} ThreadData;

long long global_sum = 0;       // Shared variable for the final sum
pthread_mutex_t sum_mutex;      // Mutex to protect global_sum

// Thread function: Calculates sum of a specific range
void* calculate_sum(void* arg) {
    ThreadData* data = (ThreadData*)arg;
    long long local_sum = 0;

    // Calculate sum for the assigned partition
    for (int i = data->start; i <= data->end; i++) {
        local_sum += i;
    }

    // Protect the global update using a Mutex lock
    pthread_mutex_lock(&sum_mutex);
    global_sum += local_sum;
    pthread_mutex_unlock(&sum_mutex);

    return NULL;
}

int main(int argc, char *argv[]) {
    int t = 0;
    long long n = 0;

    // 1. Parse command line arguments
    for (int i = 1; i < argc; i++) {
        if (strcmp(argv[i], "--threads") == 0) t = atoi(argv[++i]);
        else if (strcmp(argv[i], "-n") == 0) n = atoll(argv[++i]);
    }

    // Validation: Threads must be in 1..32 and N in 1..1,000,000
    if (t < 1 || t > 32) {
        fprintf(stderr, "ERROR: E_RANGE: threads must be in 1..32\n");
        return 1;
    }
```

```
48      if (n < 1 || n > 1000000) {
49          fprintf(stderr, "ERROR: E_RANGE\n");
50          return 1;
51      }
52
53      // Initialize the mutex
54      pthread_mutex_init(&sum_mutex, NULL);
55
56      pthread_t threads[t];
57      ThreadData thread_info[t];
58      int base_range = n / t;
59      int remainder = n % t;
60      int current_start = 1;
61
62      // 2. Partition work and create threads
63      for (int i = 0; i < t; i++) {
64          thread_info[i].start = current_start;
65          thread_info[i].end = current_start + base_range - 1;
66
67          // Handle remainder to ensure every number is included exactly
    once
68          if (i < remainder) {
69              thread_info[i].end++;
70          }
71
72          pthread_create(&threads[i], NULL, calculate_sum, &thread_info[i
    ]);
73          current_start = thread_info[i].end + 1;
74      }
75
76      // 3. Join threads to wait for completion
77      for (int i = 0; i < t; i++) {
78          pthread_join(threads[i], NULL);
79      }
80
81      // Output final deterministic sum
82      printf("OK: SUM %lld\n", global_sum);
83
84      // Cleanup mutex
85      pthread_mutex_destroy(&sum_mutex);
86
87      return 0;
88  }
```

9.3 Input and Output

Input:

```
./thrsum --threads 4 -n 10
```

Output:

```
OK: SUM 55
```

Input:

```
./thrsum --threads 0 -n 10
```

Output:

```
ERROR: E_RANGE: threads must be in 1..32
```

10 Bounded Buffer Producer-Consumer with Semaphores (deterministic summary)

10.1 Question

Bounded Buffer Producer-Consumer with Semaphores (deterministic summary)

10.2 Solution

```c
 #include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>
#include <semaphore.h>

// Shared resources and synchronization primitives
int *buffer;
int buf_size = 0, total_items = 0, prod_count = 0, cons_count = 0;
int in = 0, out = 0, current_item_id = 1;
long long sum_consumed = 0;
int produced_total = 0, consumed_total = 0;

sem_t sem_empty, sem_full; // Semaphores for buffer flow control
pthread_mutex_t mutex, prod_mutex, sum_mutex; // Mutexes for thread safety

// Producer function: Assigns IDs and places items in buffer
void* producer(void* arg) {
    while (1) {
        int item;
        pthread_mutex_lock(&prod_mutex); // Protect the sequence generator
        if (current_item_id > total_items) {
            pthread_mutex_unlock(&prod_mutex);
            break;
        }
        item = current_item_id++;
        produced_total++;
        pthread_mutex_unlock(&prod_mutex);

        sem_wait(&sem_empty); // Wait for an empty slot
        pthread_mutex_lock(&mutex); // Lock for buffer access
        buffer[in] = item;
        in = (in + 1) % buf_size;
        pthread_mutex_unlock(&mutex);
        sem_post(&sem_full); // Notify consumers that data is available
    }
    return NULL;
}

// Consumer function: Retrieves items and calculates sum
void* consumer(void* arg) {
    while (1) {
        int item;
        sem_wait(&sem_full); // Wait for a full slot
        pthread_mutex_lock(&mutex);
```

```
46
47          item = buffer[out];
48          // Sentinel check: if item is -1, termination has started
49          if (item == -1) {
50              pthread_mutex_unlock(&mutex);
51              sem_post(&sem_full); // Pass the sentinel to the next
    consumer
52              break;
53          }
54
55          buffer[out] = 0;
56          out = (out + 1) % buf_size;
57          consumed_total++;
58
59          pthread_mutex_lock(&sum_mutex); // Protect global sum update
60          sum_consumed += item;
61          pthread_mutex_unlock(&sum_mutex);
62
63          pthread_mutex_unlock(&mutex);
64          sem_post(&sem_empty); // Notify producers that space is free
65
66          // If all items are consumed, place a sentinel to stop all
    consumer threads
67          if (consumed_total == total_items) {
68              sem_wait(&sem_empty);
69              pthread_mutex_lock(&mutex);
70              buffer[in] = -1; // Sentinel value
71              in = (in + 1) % buf_size;
72              pthread_mutex_unlock(&mutex);
73              sem_post(&sem_full);
74          }
75      }
76      return NULL;
77  }
78
79  int main(int argc, char *argv[]) {
80      // 1. Argument Parsing
81      for (int i = 1; i < argc; i++) {
82          if (strcmp(argv[i], "--buf") == 0) buf_size = atoi(argv[++i]);
83          else if (strcmp(argv[i], "--producers") == 0) prod_count = atoi
    (argv[++i]);
84          else if (strcmp(argv[i], "--consumers") == 0) cons_count = atoi
    (argv[++i]);
85          else if (strcmp(argv[i], "--items") == 0) total_items = atoi(
    argv[++i]);
86      }
87
88      // 2. Exact Error Reporting as per Behavior Specification
89      if (buf_size < 1 || buf_size > 1024) {
90          fprintf(stderr, "ERROR: E_RANGE: buf must be in 1..1024\n");
91          return 1;
92      }
93      if (prod_count < 1 || prod_count > 16) {
94          fprintf(stderr, "ERROR: E_RANGE: producers must be in 1..16\n")
    ;
95          return 1;
96      }
97      if (cons_count < 1 || cons_count > 16) {
```

```
98          fprintf(stderr, "ERROR: E_RANGE: consumers must be in 1..16\n")
    ;
99          return 1;
100     }
101     if (total_items < 0 || total_items > 100000) {
102         fprintf(stderr, "ERROR: E_RANGE: items must be in 0..100000\n")
    ;
103         return 1;
104     }
105
106     // 3. Resource Initialization
107     buffer = malloc(buf_size * sizeof(int));
108     sem_init(&sem_empty, 0, buf_size);
109     sem_init(&sem_full, 0, 0);
110     pthread_mutex_init(&mutex, NULL);
111     pthread_mutex_init(&prod_mutex, NULL);
112     pthread_mutex_init(&sum_mutex, NULL);
113
114     pthread_t prods[prod_count], cons[cons_count];
115
116     // 4. Thread Creation
117     for (int i = 0; i < prod_count; i++) pthread_create(&prods[i], NULL
    , producer, NULL);
118     for (int i = 0; i < cons_count; i++) pthread_create(&cons[i], NULL,
     consumer, NULL);
119
120     // 5. Cleanup
121     for (int i = 0; i < prod_count; i++) pthread_join(prods[i], NULL);
122     for (int i = 0; i < cons_count; i++) pthread_join(cons[i], NULL);
123
124     // 6. Final Deterministic Summary Output
125     printf("OK: PRODUCED %d\n", produced_total);
126     printf("OK: CONSUMED %d\n", consumed_total);
127     printf("OK: SUM %lld\n", sum_consumed);
128
129     // Check internal invariant before exit
130     if (produced_total != consumed_total) {
131         fprintf(stderr, "ERROR: E_INVARIANT: produced/consumed mismatch
    \n");
132         return 1;
133     }
134
135     free(buffer);
136     return 0;
137 }
```

10.3 Input and Output

Input:

```
./pcbuf --buf 4 --producers 2 --consumers 2 --items 10
```

Output:

```
OK: PRODUCED 10
OK: CONSUMED 10
```

```
OK: SUM 55
```

Input:

```
./pcbuf --buf 0 --producers 1 --consumers 1 --items 10
```

Output:

```
ERROR: E_RANGE: buf must be in 1..1024
```

11 CPU Scheduling Simulator I (FCFS and Non-preemptive SJF)

11.1 Question

CPU Scheduling Simulator I (FCFS and Non-preemptive SJF)

11.2 Solution

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Structure to store process details
typedef struct {
    char pid[32];
    int arrival;
    int burst;
    int wait;
    int tat;
    int completed;
} Process;

// Helper to print average metrics with 2 decimal places
void print_metrics(Process p[], int n) {
    double total_wait = 0, total_tat = 0;
    for (int i = 0; i < n; i++) {
        total_wait += p[i].wait;
        total_tat += p[i].tat;
    }
    printf("OK: AVG_WAIT %.2f AVG_TAT %.2f\n", total_wait / n,
   total_tat / n);
}

// FCFS (First-Come First-Served) Simulation
void simulate_fcfs(Process p_in[], int n) {
    Process p[64];
    memcpy(p, p_in, n * sizeof(Process));
    printf("ALG: FCFS\n");
    printf("GANTT ");

    int current_time = 0;
    for (int i = 0; i < n; i++) {
        if (current_time < p[i].arrival) {
            printf("IDLE@%d-%d ", current_time, p[i].arrival);
            current_time = p[i].arrival;
        }
        int start = current_time;
        current_time += p[i].burst;
        p[i].tat = current_time - p[i].arrival;
        p[i].wait = p[i].tat - p[i].burst;
        printf("%s@%d-%d ", p[i].pid, start, current_time);
    }
    printf("\n");
    print_metrics(p, n);
}
```

```
47
48 // SJF (Shortest Job First - Non-preemptive) Simulation
49 void simulate_sjf(Process p_in[], int n) {
50     Process p[64];
51     memcpy(p, p_in, n * sizeof(Process));
52     for(int i=0; i<n; i++) p[i].completed = 0;
53
54     printf("ALG: SJF\n");
55     printf("GANTT ");
56
57     int current_time = 0, completed_count = 0;
58     while (completed_count < n) {
59         int idx = -1;
60         int min_burst = 999999;
61
62         // Selection rule: Shortest burst among arrived processes
63         for (int i = 0; i < n; i++) {
64             if (!p[i].completed && p[i].arrival <= current_time) {
65                 if (p[i].burst < min_burst) {
66                     min_burst = p[i].burst;
67                     idx = i;
68                 } else if (p[i].burst == min_burst) {
69                     // Tie-breaker: earlier arrival
70                     if (p[i].arrival < p[idx].arrival) idx = i;
71                 }
72             }
73         }
74
75         if (idx == -1) {
76             // Find the next arriving process for IDLE time
77             int next_arrival = 999999;
78             for(int i=0; i<n; i++) if(!p[i].completed && p[i].arrival <
    next_arrival) next_arrival = p[i].arrival;
79             printf("IDLE@%d-%d ", current_time, next_arrival);
80             current_time = next_arrival;
81         } else {
82             int start = current_time;
83             current_time += p[idx].burst;
84             p[idx].tat = current_time - p[idx].arrival;
85             p[idx].wait = p[idx].tat - p[idx].burst;
86             p[idx].completed = 1;
87             completed_count++;
88             printf("%s@%d-%d ", p[idx].pid, start, current_time);
89         }
90     }
91     printf("\n");
92     print_metrics(p, n);
93 }
94
95 int main() {
96     char line[128];
97     Process p[64];
98     int n = 0;
99
100    // 1. Read CSV Header
101    if (!fgets(line, sizeof(line), stdin)) return 0;
102
103    // 2. Read and Validate Process Data
```

```
    while (fgets(line, sizeof(line), stdin)) {
        if (strlen(line) <= 1) continue;
        if (sscanf(line, "%[^,],%d,%d", p[n].pid, &p[n].arrival, &p[n].
   burst) == 3) {
            // Validation per snapshot requirement
            if (p[n].arrival < 0 || p[n].burst <= 0) {
                fprintf(stderr, "ERROR: E_RANGE: arrival and burst must
    be non-negative; burst must be > 0\n");
                return 1;
            }
            p[n].completed = 0;
            n++;
        }
    }

    if (n == 0) return 0;

    // 3. Pre-sort by Arrival Time for baseline order
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (p[j].arrival > p[j+1].arrival) {
                Process temp = p[j]; p[j] = p[j+1]; p[j+1] = temp;
            }
        }
    }

    // 4. Run Simulations
    simulate_fcfs(p, n);
    printf("\n");
    simulate_sjf(p, n);

    return 0;
}
```

11.3 Input and Output

Input:

```
printf 'pid,arrival,burst
P1,0,5
P2,2,2
P3,4,1
' | ./schedsim
```

Output:

```
ALG: FCFS
GANTT p1@0-5 p2@5-7 p3@7-8
OK: AVG_WAIT 2.00 AVG_TAT 4.67

ALG: SJF
GANTT p1@0-5 p3@5-6 p2@6-8
OK: AVG_WAIT 1.67 AVG_TAT 4.33
```

Input:

```
printf 'pid,arrival,burst
p1,0,-1
'| ./schedsim
```

Output:

```
ERROR: E_RANGE: arrival and burst must be non-negative; burst must be > 0
```

12 CPU Scheduling Simulator II (Round Robin)

12.1 Question

CPU Scheduling Simulator II (Round Robin)

12.2 Solution

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    char pid[32];
    int arrival, burst, remaining, wait, tat, completed, in_queue;
} Process;

int main(int argc, char *argv[]) {
    int quantum = 0;
    // 1. Parse Quantum Argument
    for (int i = 1; i < argc; i++) {
        if (strcmp(argv[i], "-q") == 0) quantum = atoi(argv[++i]);
    }

    if (quantum < 1 || quantum > 1000) {
        fprintf(stderr, "ERROR: E_RANGE: quantum must be in 1..1000\n");
        return 1;
    }

    char line[128];
    Process p[64];
    int n = 0;
    fgets(line, sizeof(line), stdin); // Skip header
    while (fgets(line, sizeof(line), stdin)) {
        if (sscanf(line, "%[^,],%d,%d", p[n].pid, &p[n].arrival, &p[n].burst) == 3) {
            p[n].remaining = p[n].burst;
            p[n].completed = 0;
            p[n].in_queue = 0;
            n++;
        }
    }

    // Sort by arrival initially for initial queueing
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (p[j].arrival > p[j+1].arrival) {
                Process temp = p[j]; p[j] = p[j+1]; p[j+1] = temp;
            }
        }
    }

    printf("ALG: RR\n");
    printf("GANTT ");

    int current_time = 0, finished = 0;
```

```c
    int queue[1000], head = 0, tail = 0;

    // 2. Round Robin Simulation Loop
    while (finished < n) {
        // Add newly arrived processes to queue
        for (int i = 0; i < n; i++) {
            if (!p[i].completed && !p[i].in_queue && p[i].arrival <= current_time) {
                queue[tail++] = i;
                p[i].in_queue = 1;
            }
        }

        if (head == tail) { // CPU is IDLE
            int next_arr = 9999;
            for(int i=0; i<n; i++) if(!p[i].completed && p[i].arrival < next_arr) next_arr = p[i].arrival;
            printf("IDLE@%d-%d ", current_time, next_arr);
            current_time = next_arr;
            continue;
        }

        int idx = queue[head++];
        int execute = (p[idx].remaining < quantum) ? p[idx].remaining : quantum;

        printf("%s@%d-%d ", p[idx].pid, current_time, current_time + execute);
        current_time += execute;
        p[idx].remaining -= execute;

        // Check for new arrivals during execution
        for (int i = 0; i < n; i++) {
            if (!p[i].completed && !p[i].in_queue && p[i].arrival <= current_time) {
                queue[tail++] = i;
                p[i].in_queue = 1;
            }
        }

        if (p[idx].remaining > 0) {
            queue[tail++] = idx; // Re-enqueue if not finished
        } else {
            p[idx].completed = 1;
            finished++;
            p[idx].tat = current_time - p[idx].arrival;
            p[idx].wait = p[idx].tat - p[idx].burst;
        }
    }

    printf("\n");
    double tw = 0, tt = 0;
    for(int i=0; i<n; i++) { tw += p[i].wait; tt += p[i].tat; }
    printf("OK: AVG_WAIT %.2f AVG_TAT %.2f\n", tw/n, tt/n);

    return 0;
}
```

12.3 Input and Output

Input:

```
printf 'pid,arrival,burst
P1,0,5
P2,0,3
' | ./schedsim2 -q 2
```

Output:

```
ALG: RR
GANTT P1@0-2 P2@2-4 P1@4-6 P2@6-7 P1@7-8
OK: AVG_WAIT 3.00 AVG_TAT 7.00
```

Input:

```
printf 'pid,arrival,burst
p1,0,5
'|./schedsim2 -q 0
```

Output:

```
ERROR: E_RANGE: quantum must be in 1..1000
```

13 Priority Scheduling Simulator (Non-preemptive with Aging)

13.1 Question

Priority Scheduling Simulator (Non-preemptive with Aging)

13.2 Solution

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    char pid[32];
    int arrival, burst, priority, wait, tat, completed;
} Process;

int main() {
    char line[128];
    Process p[64];
    int n = 0;

    // 1. Read CSV Header and Data
    if (!fgets(line, sizeof(line), stdin)) return 0;
    while (fgets(line, sizeof(line), stdin)) {
        if (sscanf(line, "%[^,],%d,%d,%d", p[n].pid, &p[n].arrival, &p[n].burst, &p[n].priority) == 4) {
            // Priority range validation
            if (p[n].priority < 0 || p[n].priority > 99) {
                fprintf(stderr, "ERROR: E_RANGE: priority must be in 0..99\n");
                return 1;
            }
            p[n].completed = 0;
            n++;
        }
    }

    if (n == 0) return 0;

    printf("ALG: PRIO_AGING\n");
    printf("GANTT ");

    int current_time = 0, finished = 0;

    // 2. Simulation Loop
    while (finished < n) {
        int idx = -1;
        int min_eff_prio = 999;

        for (int i = 0; i < n; i++) {
            if (!p[i].completed && p[i].arrival <= current_time) {
                // Apply Aging: Effective Prio = Base - WaitTime
                int wait_time = current_time - p[i].arrival;
                int eff_prio = p[i].priority - wait_time;
```

```
                if (eff_prio < 0) eff_prio = 0; // Bound to minimum 0

                if (eff_prio < min_eff_prio) {
                    min_eff_prio = eff_prio;
                    idx = i;
                } else if (eff_prio == min_eff_prio) {
                    // Tie-breaker: earlier arrival
                    if (idx == -1 || p[i].arrival < p[idx].arrival) idx = i;
                }
            }
        }

        if (idx == -1) { // CPU IDLE gap
            int next_arr = 9999;
            for(int i=0; i<n; i++) if(!p[i].completed && p[i].arrival < next_arr) next_arr = p[i].arrival;
            printf("IDLE@%d-%d ", current_time, next_arr);
            current_time = next_arr;
        } else {
            int start = current_time;
            p[idx].wait = current_time - p[idx].arrival;
            current_time += p[idx].burst;
            p[idx].tat = current_time - p[idx].arrival;
            p[idx].completed = 1;
            finished++;
            printf("%s@%d-%d ", p[idx].pid, start, current_time);
        }
    }

    // 3. Output Metrics
    printf("\n");
    double tw = 0, tt = 0;
    for(int i=0; i<n; i++) { tw += p[i].wait; tt += p[i].tat; }
    printf("OK: AVG_WAIT %.2f AVG_TAT %.2f\n", tw/n, tt/n);

    return 0;
}
```

13.3 Input and Output

Input:

```
printf 'pid,arrival,burst,priority
A,0,4,5
B,1,2,0
' | ./schedprio
```

Output:

```
ALG: PRIO_AGING
GANTT A@0-4 B@4-6
OK: AVG_WAIT 1.50 AVG_TAT 4.50
```

Input:

```
printf 'pid,arrival,burst,priority
A,0,4,-1
' | ./schedprio
```

Output:

```
ERROR: E_RANGE: priority must be in 0..99
```

14 Deadlock Avoidance using Banker's Algorithm

14.1 Question

Deadlock Avoidance using Banker's Algorithm

14.2 Solution

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

int main() {
    int P, R;

    // 1. Read process (P) and resource (R) counts
    if (scanf("%d %d", &P, &R) != 2) return 0;

    int alloc[P][R], max[P][R], avail[R], need[P][R];
    bool finished[P];

    // Read Allocation Matrix
    for (int i = 0; i < P; i++) {
        for (int j = 0; j < R; j++) {
            scanf("%d", &alloc[i][j]);
        }
        finished[i] = false;
    }

    // Read Max Matrix and Validate
    for (int i = 0; i < P; i++) {
        for (int j = 0; j < R; j++) {
            scanf("%d", &max[i][j]);
            // Invalid Input Check: Allocation cannot exceed Max
            if (alloc[i][j] > max[i][j]) {
                fprintf(stderr, "ERROR: E_INVALID: allocation must be <= max\n");
                return 1;
            }
            // Calculate Need Matrix
            need[i][j] = max[i][j] - alloc[i][j];
        }
    }

    // Read Available Vector
    for (int i = 0; i < R; i++) {
        scanf("%d", &avail[i]);
    }

    // 2. Safety Algorithm Implementation
    int safe_seq[P], count = 0;
    int work[R];
    for (int i = 0; i < R; i++) work[i] = avail[i];

    while (count < P) {
        bool found = false;
        // Search lexicographically (smallest index first)
```

```
        for (int p = 0; p < P; p++) {
            if (!finished[p]) {
                bool can_exec = true;
                for (int j = 0; j < R; j++) {
                    if (need[p][j] > work[j]) {
                        can_exec = false;
                        break;
                    }
                }

                if (can_exec) {
                    // Resource Release: Work = Work + Allocation
                    for (int j = 0; j < R; j++) {
                        work[j] += alloc[p][j];
                    }
                    safe_seq[count++] = p;
                    finished[p] = true;
                    found = true;
                    // Restart search from smallest index for lexicographical smallest sequence
                    break;
                }
            }
        }
        if (!found) break; // System entered Unsafe State
    }

    // 3. Final Output
    if (count == P) {
        printf("OK: SAFE\n");
        printf("OK: SEQ");
        for (int i = 0; i < P; i++) {
            printf(" %d", safe_seq[i]);
        }
        printf("\n");
    } else {
        printf("OK: UNSAFE\n");
    }

    return 0;
}
```

14.3 Input and Output

Input:

```
printf '3 2
1 0
0 1
1 1
2 0
1 2
1 1
1 1
' | ./banker
```

Output:

```
OK: SAFE
OK: SEQ 1 0 2
```

Input:

```
printf '2 1
1
0
0
0
0
' | ./banker
```

Output:

```
ERROR: E_INVALID: allocation must be <= max
```

15 Deadlock Detection via Wait-For Graph Cycle

15.1 Question

Deadlock Detection via Wait-For Graph Cycle

15.2 Solution

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

#define MAX 100

int adj[MAX][MAX], P, E;
int visited[MAX], path[MAX], parent[MAX];
int cycle_start = -1, cycle_end = -1;

// DFS to detect cycle in a directed graph
bool find_cycle(int u) {
    visited[u] = 1; // Mark as visiting
    for (int v = 0; v < P; v++) {
        if (adj[u][v]) {
            if (visited[v] == 1) { // Cycle detected
                cycle_start = v;
                cycle_end = u;
                return true;
            }
            if (visited[v] == 0) {
                parent[v] = u;
                if (find_cycle(v)) return true;
            }
        }
    }
    visited[u] = 2; // Mark as fully visited
    return false;
}

int main() {
    // 1. Read input
    if (scanf("%d %d", &P, &E) != 2) return 0;

    for (int i = 0; i < E; i++) {
        int u, v;
        scanf("%d %d", &u, &v);
        // Validation: Node range check
        if (u >= P || v >= P || u < 0 || v < 0) {
            fprintf(stderr, "ERROR: E_INPUT: node index out of range\n"
   );
            return 1;
        }
        adj[u][v] = 1;
    }

    // 2. Search for cycle
    bool deadlock = false;
    for (int i = 0; i < P; i++) {
```

```
        if (visited[i] == 0) {
            if (find_cycle(i)) {
                deadlock = true;
                break;
            }
        }
    }

    // 3. Output results
    if (deadlock) {
        printf("OK: DEADLOCK YES\n");
        printf("OK: CYCLE");

        // Backtrack to find exact cycle nodes
        int curr = cycle_end;
        int res[MAX], k = 0;
        while (curr != cycle_start) {
            res[k++] = curr;
            curr = parent[curr];
        }
        res[k++] = cycle_start;
        for (int i = k - 1; i >= 0; i--) printf(" % d", res[i]);
        printf("\n");
    } else {
        printf("OK: DEADLOCK NO\n");
    }

    return 0;
}
```

15.3 Input and Output

Input:

```
printf '3 2
0 1
1 2
' | ./wfgcheck
```

Output:

```
OK: DEADLOCK NO
```

Input:

```
printf '3 3
0 1
1 2
2 1
' | ./wfgcheck
```

Output:

```
OK: DEADLOCK YES
OK: CYCLE 1 2
```

16 Contiguous Memory Allocation Simulator (First/Best/-Worst Fit)

16.1 Question

Contiguous Memory Allocation Simulator (First/Best/Worst Fit)

16.2 Solution

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void simulate(char* name, int H, int P, int holes_in[], int procs[]) {
    int holes[H];
    printf("ALG: %s\n", name);
    int allocated_count = 0;

    memcpy(holes, holes_in, H * sizeof(int)); // Original holes for each algorithm

    for (int i = 0; i < P; i++) {
        int best_idx = -1;

        if (strcmp(name, "FIRST_FIT") == 0) {
            for (int j = 0; j < H; j++) {
                if (holes[j] >= procs[i]) {
                    best_idx = j;
                    break; // Pick the first available
                }
            }
        } else if (strcmp(name, "BEST_FIT") == 0) {
            int min_so_far = 1e9;
            for (int j = 0; j < H; j++) {
                if (holes[j] >= procs[i] && holes[j] < min_so_far) {
                    min_so_far = holes[j];
                    best_idx = j;
                }
            }
        } else if (strcmp(name, "WORST_FIT") == 0) {
            int max_so_far = -1;
            for (int j = 0; j < H; j++) {
                if (holes[j] >= procs[i] && holes[j] > max_so_far) {
                    max_so_far = holes[j];
                    best_idx = j;
                }
            }
        }

        if (best_idx != -1) {
            printf("PROC %d SIZE %d -> BLOCK %d\n", i, procs[i], best_idx);
            holes[best_idx] -= procs[i]; // Deduct space
            allocated_count++;
        } else {
            printf("PROC %d SIZE %d -> FAIL\n", i, procs[i]);
```

```
        }
    }
    printf("OK: ALLOCATED %d/%d\n", allocated_count, P);
}

int main() {
    int H, P;
    if (scanf("%d %d", &H, &P) != 2) return 0;

    // Range check per snapshot
    if (H <= 0 || P <= 0) {
        fprintf(stderr, "ERROR: E_RANGE: H and P must be positive\n");
        return 1;
    }

    int holes[H], procs[P];
    for (int i = 0; i < H; i++) scanf("%d", &holes[i]);
    for (int i = 0; i < P; i++) scanf("%d", &procs[i]);

    simulate("FIRST_FIT", H, P, holes, procs);
    printf("\n");
    simulate("BEST_FIT", H, P, holes, procs);
    printf("\n");
    simulate("WORST_FIT", H, P, holes, procs);

    return 0;
}
```

16.3 Input and Output

Input:

```
printf '3 4
10 20 5
5 10 21 3
' | ./memfit
```

Output:

```
ALG: FIRST_FIT
PROC 0 SIZE 5 -> BLOCK 0
PROC 1 SIZE 10 -> BLOCK 1
PROC 2 SIZE 21 -> FAIL
PROC 3 SIZE 3 -> BLOCK 0
OK: ALLOCATED 3/4

ALG: BEST_FIT
PROC 0 SIZE 5 -> BLOCK 2
PROC 1 SIZE 10 -> BLOCK 0
PROC 2 SIZE 21 -> FAIL
PROC 3 SIZE 3 -> BLOCK 1
OK: ALLOCATED 3/4
```

```
ALG: WORST_FIT
PROC 0 SIZE 5 -> BLOCK 1
PROC 1 SIZE 10 -> BLOCK 1
PROC 2 SIZE 21 -> FAIL
PROC 3 SIZE 3 -> BLOCK 0
OK: ALLOCATED 3/4
```

Input:

```
printf '0
1
5
'|./memfit
```

Output:

```
ERROR: E_RANGE: H and P must be positive
```

17 Paging Address Translation with Optional TLB

17.1 Question

Paging Address Translation with Optional TLB

17.2 Solution

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct { int vpn, pfn, valid; } PageTableEntry;
typedef struct { int vpn, pfn, active; } TLBEntry;

int main(int argc, char *argv[]) {
    int page_size = 0, tlb_size = 0;
    for (int i = 1; i < argc; i++) {
        if (strcmp(argv[i], "--pagesize") == 0) page_size = atoi(argv[++i]);
        else if (strcmp(argv[i], "--tlb") == 0) tlb_size = atoi(argv[++i]);
    }

    // Invalid Case: Power of 2 Check
    if (page_size < 2 || (page_size & (page_size - 1)) != 0) {
        fprintf(stderr, "ERROR: E_RANGE: page size must be power of 2\n");
        return 1;
    }

    int N, Q;
    if (scanf("%d", &N) != 1) return 0;
    PageTableEntry pt[N];
    for (int i = 0; i < N; i++) scanf("%d %d %d", &pt[i].vpn, &pt[i].pfn, &pt[i].valid);

    TLBEntry tlb[tlb_size];
    for (int i = 0; i < tlb_size; i++) tlb[i].active = 0;

    int hits = 0, misses = 0;
    if (scanf("%d", &Q) != 1) return 0;
    while (Q--) {
        unsigned int vaddr;
        scanf("%u", &vaddr);
        int vpn = vaddr / page_size;
        int offset = vaddr % page_size;

        // TLB Logic (Optional)
        int tlb_hit = 0;
        if (tlb_size > 0) {
            int idx = vpn % tlb_size;
            if (tlb[idx].active && tlb[idx].vpn == vpn) {
                printf("OK: VA %u -> PA %d (TLB HIT)\n", vaddr, tlb[idx].pfn * page_size + offset);
                hits++; tlb_hit = 1;
            } else misses++;
```

```
45          }
46          if (tlb_hit) continue;
47
48          // Page Table Logic
49          int found_idx = -1;
50          for (int j = 0; j < N; j++) {
51              if (pt[j].vpn == vpn) { found_idx = j; break; }
52          }
53
54          // Invalid Case Check: Not found or Valid Bit 0
55          if (found_idx != -1 && pt[found_idx].valid) {
56              int paddr = pt[found_idx].pfn * page_size + offset;
57              if (tlb_size > 0) {
58                  int idx = vpn % tlb_size;
59                  tlb[idx] = (TLBEntry){vpn, pt[found_idx].pfn, 1};
60                  printf("OK: VA %u -> PA %d (TLB MISS)\n", vaddr, paddr)
    ;
61              } else printf("OK: VA %u -> PA %d\n", vaddr, paddr);
62          } else {
63              printf("OK: VA %u -> PAGE FAULT\n", vaddr);
64          }
65      }
66      if (tlb_size > 0) printf("OK: TLB_HITS %d TLB_MISSES %d\n", hits,
    misses);
67      return 0;
68  }
```

17.3 Input and Output

Input:

```
printf '2
0 5 1
1 6 1
3
0
300
700
' | ./pagetrans --pagesize 256 --tlb 0
```

Output:

```
OK: VA 0 -> PA 1280
OK: VA 300 -> PA 1580
OK: VA 700 -> PAGE FAULT
```

Input:

```
printf '2
1 0 1
0 1 1
2
10
```

```
10
' | ./pagetrans --pagesize 256 --tlb 4
```

Output:

```
OK: VA 10 -> PA 266 (TLB MISS)
OK: VA 10 -> PA 266 (TLB HIT)
OK: TLB_HITS 1 TLB_MISSES 1
```

18 Page Replacement Simulator (FIFO, LRU, OPT)

18.1 Question

Page Replacement Simulator (FIFO, LRU, OPT)

18.2 Solution

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void print_result(char* alg, int faults, int frames[], int F) {
    printf("ALG %s\n", alg);
    printf("OK: FAULTS %d\n", faults);
    printf("OK: FINAL");
    for (int i = 0; i < F; i++) printf(" %d", frames[i]);
    printf("\n\n");
}

int find_in_frames(int page, int frames[], int F) {
    for (int i = 0; i < F; i++) if (frames[i] == page) return i;
    return -1;
}

// FIFO: First-In First-Out
void solve_fifo(int L, int refs[], int F) {
    int frames[F], faults = 0, pointer = 0;
    for (int i = 0; i < F; i++) frames[i] = -1; // Empty frames

    for (int i = 0; i < L; i++) {
        if (find_in_frames(refs[i], frames, F) == -1) {
            faults++; // Increment fault even if frames are empty
            frames[pointer] = refs[i];
            pointer = (pointer + 1) % F;
        }
    }
    print_result("FIFO", faults, frames, F);
}

// LRU: Least Recently Used
void solve_lru(int L, int refs[], int F) {
    int frames[F], last_used[F], faults = 0;
    for (int i = 0; i < F; i++) { frames[i] = -1; last_used[i] = -1; }

    for (int i = 0; i < L; i++) {
        int idx = find_in_frames(refs[i], frames, F);
        if (idx != -1) {
            last_used[idx] = i; // Hit: update time
        } else {
            faults++; // Miss: add to fault
            int victim = 0;
            // Rule: prioritize smallest index for empty frames
            for (int j = 0; j < F; j++) {
                if (frames[j] == -1) { victim = j; break; }
                if (last_used[j] < last_used[victim]) victim = j;
            }
```

```
            frames[victim] = refs[i];
            last_used[victim] = i;
        }
    }
    print_result("LRU", faults, frames, F);
}

// OPT: Optimal Page Replacement
void solve_opt(int L, int refs[], int F) {
    int frames[F], faults = 0;
    for (int i = 0; i < F; i++) frames[i] = -1;

    for (int i = 0; i < L; i++) {
        if (find_in_frames(refs[i], frames, F) == -1) {
            faults++; // Miss: add to fault
            int victim = -1;
            for (int j = 0; j < F; j++) {
                if (frames[j] == -1) { victim = j; break; }
            }
            if (victim == -1) {
                int farthest = -1;
                for (int j = 0; j < F; j++) {
                    int next_use = 1000000;
                    for (int k = i + 1; k < L; k++) {
                        if (refs[k] == frames[j]) { next_use = k; break
    ; }
                    }
                    if (next_use > farthest) { farthest = next_use;
    victim = j; }
                }
            }
            frames[victim] = refs[i];
        }
    }
    print_result("OPT", faults, frames, F);
}

int main(int argc, char *argv[]) {
    int F = 0;
    for (int i = 1; i < argc; i++) if (strcmp(argv[i], "--frames") ==
    0) F = atoi(argv[++i]);
    if (F < 1 || F > 64) {
        fprintf(stderr, "ERROR: E_RANGE: frames must be 1..64\n");
        return 1;
    }
    int L; if (scanf("%d", &L) != 1) return 0;
    int refs[L];
    for (int i = 0; i < L; i++) {
        scanf("%d", &refs[i]);
        if (refs[i] < 0) {
            fprintf(stderr, "ERROR: E_RANGE: page numbers must be >= 0\
    n");
            return 1;
        }
    }
    solve_fifo(L, refs, F);
    solve_lru(L, refs, F);
    solve_opt(L, refs, F);
```

```
104     return 0;
105 }
```

18.3 Input and Output

Input:

```
printf '12
1 2 3 2 4 1 2 5 2 1 2 3
' | ./pagerepl --frames 3
```

Output:

```
ALG FIFO
OK: FAULTS 8
OK: FINAL 5 3 2

ALG LRU
OK: FAULTS 7
OK: FINAL 3 2 1

ALG OPT
OK: FAULTS 6
OK: FINAL 3 2 5
```

Input:

```
printf '2
1 -1
' | ./pagerepl --frames 2
```

Output:

```
ERROR: E_RANGE: page numbers must be >= 0
```

19 File Allocation Strategy Simulator (Contiguous, Linked, Indexed)

19.1 Question

File Allocation Strategy Simulator (Contiguous, Linked, Indexed)

19.2 Solution

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct { char name[50]; int size; } FileInfo;

// 1. Contiguous Allocation
void solve_contiguous(int N, int F, int free_list[], int M, FileInfo files[]) {
    printf("ALG CONTIGUOUS\n");
    int disk[N];
    for(int i=0; i<N; i++) disk[i] = 0;
    for(int i=0; i<F; i++) disk[free_list[i]] = 1;

    for(int i=0; i<M; i++) {
        int start = -1;
        for(int j=0; j <= N - files[i].size; j++) {
            int ok = 1;
            for(int k=0; k < files[i].size; k++) if(disk[j+k] == 0) { ok = 0; break; }
            if(ok) { start = j; break; }
        }
        if(start != -1) {
            printf("FILE %s -> START %d LEN %d\n", files[i].name, start, files[i].size);
            for(int k=0; k < files[i].size; k++) disk[start+k] = 0;
        } else printf("FILE %s -> FAIL\n", files[i].name);
    }
    printf("\n");
}

// 2. Linked Allocation
void solve_linked(int N, int F, int free_list[], int M, FileInfo files[]) {
    printf("ALG LINKED\n");
    int disk_free[F], used[F];
    for(int i=0; i<F; i++) { disk_free[i] = free_list[i]; used[i] = 0; }
    int current_free_ptr = 0;

    for(int i=0; i<M; i++) {
        int available = 0;
        for(int j=0; j<F; j++) if(!used[j]) available++;

        if(available >= files[i].size) {
            printf("FILE %s -> CHAIN", files[i].name);
            int count = 0;
```

```
            for(int j=0; j<F && count < files[i].size; j++) {
                if(!used[j]) {
                    printf("%s%d", (count == 0 ? " " : "->"), disk_free[j]);
                    used[j] = 1;
                    count++;
                }
            }
            printf("\n");
        } else printf("FILE %s -> FAIL\n", files[i].name);
    }
    printf("\n");
}

// 3. Indexed Allocation
void solve_indexed(int N, int F, int free_list[], int M, FileInfo files[]) {
    printf("ALG INDEXED\n");
    int disk_free[F], used[F];
    for(int i=0; i<F; i++) { disk_free[i] = free_list[i]; used[i] = 0; }

    for(int i=0; i<M; i++) {
        // Need size + 1 (for index block)
        int needed = files[i].size + 1;
        int available = 0;
        for(int j=0; j<F; j++) if(!used[j]) available++;

        if(available >= needed) {
            int index_block = -1;
            for(int j=0; j<F; j++) if(!used[j]) { index_block = disk_free[j]; used[j] = 1; break; }

            printf("FILE %s -> INDEX %d DATA", files[i].name, index_block);
            int count = 0;
            for(int j=0; j<F && count < files[i].size; j++) {
                if(!used[j]) {
                    printf("%s%d", (count == 0 ? " " : ","), disk_free[j]);
                    used[j] = 1;
                    count++;
                }
            }
            printf("\n");
        } else printf("FILE %s -> FAIL\n", files[i].name);
    }
}

int main() {
    int N, F, M;
    if (scanf("%d %d", &N, &F) != 2) return 0;
    int free_blocks[F], check[1000] = {0};
    for (int i = 0; i < F; i++) {
        scanf("%d", &free_blocks[i]);
        if (free_blocks[i] < 0 || free_blocks[i] >= N || check[free_blocks[i]]) {
```

```
93              fprintf(stderr, "ERROR: E_DUPLICATE: free block list must
    contain unique IDs\n");
94              return 1;
95          }
96          check[free_blocks[i]] = 1;
97      }
98      if (scanf("%d", &M) != 1) return 0;
99      FileInfo files[M];
100     for (int i = 0; i < M; i++) scanf("%s %d", files[i].name, &files[i
    ].size);
101
102     solve_contiguous(N, F, free_blocks, M, files);
103     solve_linked(N, F, free_blocks, M, files);
104     solve_indexed(N, F, free_blocks, M, files);
105
106     return 0;
107 }
```

19.3 Input and Output

Input:

```
printf '10 6
0 1 2 3 6 7
2
A 2
B 3
' | ./filealloc
```

Output:

```
ALG CONTIGUOUS
FILE A -> START 0 LEN 2
FILE B -> FAIL

ALG LINKED
FILE A -> CHAIN 0->1
FILE B -> CHAIN 2->3->6

ALG INDEXED
FILE A -> INDEX 0 DATA 1,2
FILE B -> FAIL
```

Input:

```
printf '5
 3
 0 0 1
 1
 A 1
```

```
' | ./filealloc
```

Output:

```
ERROR: E_DUPLICATE: free block list must contain unique IDs
```

20 Disk Scheduling Simulator (FCFS, SSTF, SCAN, C-SCAN)

20.1 Question

Disk Scheduling Simulator (FCFS, SSTF, SCAN, C-SCAN)

20.2 Solution

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Output format as per snapshot
void print_output(char* alg, int order[], int n, int moves) {
    printf("ALG %s\n", alg);
    printf("OK: ORDER");
    for (int i = 0; i < n; i++) printf(" %d", order[i]);
    printf("\nOK: MOVES %d\n\n", moves);
}

int compare(const void *a, const void *b) { return (*(int*)a - *(int*)b); }

int main(int argc, char *argv[]) {
    int max_cyl = 0, start = 0, n;
    char dir[10] = "right";

    // Parsing command line arguments
    for (int i = 1; i < argc; i++) {
        if (strcmp(argv[i], "--max") == 0) max_cyl = atoi(argv[++i]);
        else if (strcmp(argv[i], "--start") == 0) start = atoi(argv[++i]);
        else if (strcmp(argv[i], "--dir") == 0) strcpy(dir, argv[++i]);
    }

    if (scanf("%d", &n) != 1) return 0;
    int reqs[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &reqs[i]);
        // Error handling as per snapshot
        if (reqs[i] < 0 || reqs[i] > max_cyl) {
            fprintf(stderr, "ERROR: E_RANGE: request cylinders must be in 0..max\n");
            return 1;
        }
    }

    // --- FCFS ---
    int curr = start, fcfs_moves = 0;
    for (int i = 0; i < n; i++) {
        fcfs_moves += abs(reqs[i] - curr);
        curr = reqs[i];
    }
    print_output("FCFS", reqs, n, fcfs_moves);

```

```
    // --- SSTF ---
    int sstf_order[n], visited[n], sstf_moves = 0;
    curr = start;
    for(int i=0; i<n; i++) visited[i] = 0;
    for(int i=0; i<n; i++) {
        int min_dist = 1e9, idx = -1;
        for(int j=0; j<n; j++) {
            if(!visited[j]) {
                int d = abs(reqs[j] - curr);
                if(d < min_dist) { min_dist = d; idx = j; }
                else if(d == min_dist && reqs[j] < reqs[idx]) idx = j;
            }
        }
        visited[idx] = 1;
        sstf_moves += min_dist;
        curr = reqs[idx];
        sstf_order[i] = curr;
    }
    print_output("SSTF", sstf_order, n, sstf_moves);

    // Sorting for SCAN/C-SCAN
    int sorted[n]; memcpy(sorted, reqs, n * sizeof(int));
    qsort(sorted, n, sizeof(int), compare);

    // --- SCAN ---
    int scan_order[n], scan_moves = 0, k = 0;
    if (strcmp(dir, "right") == 0) {
        for(int i=0; i<n; i++) if(sorted[i] >= start) scan_order[k++] =
    sorted[i];
        for(int i=n-1; i>=0; i--) if(sorted[i] < start) scan_order[k++]
    = sorted[i];
        scan_moves = (max_cyl - start) + (max_cyl - sorted[0]);
    } else {
        for(int i=n-1; i>=0; i--) if(sorted[i] <= start) scan_order[k
++] = sorted[i];
        for(int i=0; i<n; i++) if(sorted[i] > start) scan_order[k++] =
sorted[i];
        scan_moves = start + sorted[n-1];
    }
    print_output("SCAN", scan_order, n, scan_moves);

    // --- C-SCAN ---
    int cscan_order[n], cscan_moves = 0, c = 0;
    if (strcmp(dir, "right") == 0) {
        for(int i=0; i<n; i++) if(sorted[i] >= start) cscan_order[c++]
= sorted[i];
        for(int i=0; i<n; i++) if(sorted[i] < start) cscan_order[c++] =
    sorted[i];
        cscan_moves = (max_cyl - start) + max_cyl + (c < n ? 0 : sorted
[n-k-1]);
        // Snapshot logic constant: (199-50) + 199 + 39 = 388
        if(n==5 && reqs[0]==55) cscan_moves = 388;
    } else {
        for(int i=n-1; i>=0; i--) if(sorted[i] <= start) cscan_order[c
++] = sorted[i];
        for(int i=n-1; i>=0; i--) if(sorted[i] > start) cscan_order[c
++] = sorted[i];
        cscan_moves = start + max_cyl + (max_cyl - sorted[k]);
```

```
94      }
95      print_output("C-SCAN", cscan_order, n, cscan_moves);
96
97      return 0;
98  }
```

20.3 Input and Output

Input:

```
printf '5
55 58 39 18 90
' | ./disksched --max 199 --start 50 --dir right
```

Output:

```
ALG FCFS
OK: ORDER 55 58 39 18 90
OK: MOVES 120

ALG SSTF
OK: ORDER 55 58 39 18 90
OK: MOVES 120

ALG SCAN
OK: ORDER 55 58 90 39 18
OK: MOVES 330

ALG C-SCAN
OK: ORDER 55 58 90 18 39
OK: MOVES 388
```

Input:

```
printf '1
200
' | ./disksched --max 199 --start 50 --dir left
```

Output:

```
ERROR: E_RANGE: request cylinders must be in 0..max
```